**WEEK 1 RECONNAISSANCE REPORT**

**Project Title:** Automated Web Application Security Assessment for Financial Platforms

**Target Application:** OWASP Juice Shop

**Student:** Robert Mayonde

**Programme:** Cybersecurity

**Institution:** JPR Solutions

**Reporting Period:** Week 1

---

## 1. Introduction

The reconnaissance phase is an important initial stage of the Automated Web Application Security Assessment for Financial Platforms project. Its purpose is to understand the structure of a web application, identify its entry points, observe HTTP communication and establish an initial inventory of API endpoints.

OWASP Juice Shop was selected as the target application because it provides a deliberately vulnerable web application suitable for security, education and testing in an authorised local environment.

During this phase, the application was accessed through a web browser, and Chrome Developer Tools was used to observe HTTP requests generated by normal user interactions. A custom Python scanner was also used to collect initial information about pages, forms, input points and potential API endpoints.

The findings from this phase will guide subsequent development and testing of the automated security assessment tool.

## 2. Objectives

The objectives of the reconnaissance phase were to:

1. Establish a working local testing environment using Docker.

2. Identify the main pages and user input points within OWASP Juice Shop.
3. Observe HTTP requests generated by selected application features.
4. Identify API endpoints and document their observed behaviour.
5. Determine the application's technology stack using available evidence.
6. Understand the high-level structure of the application.
7. Compare browser observations with the output of the custom Python scanner.

**3. Lab Environment**

The reconnaissance activities were conducted using the following environment.

| Component | Description |
|---|---|
| Operating system | Windows |
| Virtualization software | VMware, available in the lab setup |
| Container platform | Docker |
| Target application | OWASP Juice Shop |
| Browser | Google Chrome |
| Browser inspection tool | Chrome Developer Tools |
| Assessment tool | Custom Python reconnaissance scanner |
| Report format | JSON |

The local application was accessed through its configured browser address. Docker was used to run the target application in the testing environment.

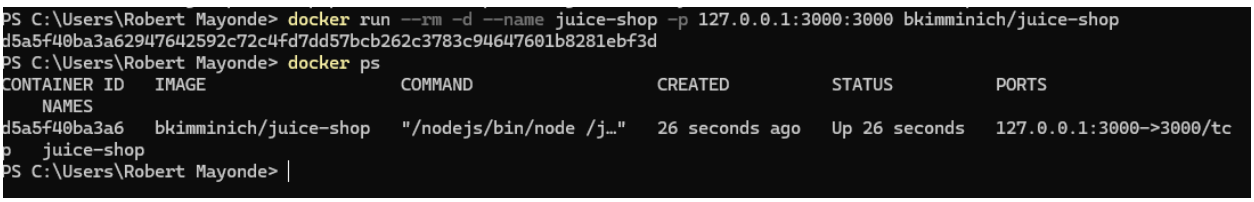


*Figure 1: OWASP Juice Shop Running in the Local Testing Environment.*

The screenshot demonstrates that the application was accessible through the browser and ready for reconnaissance.

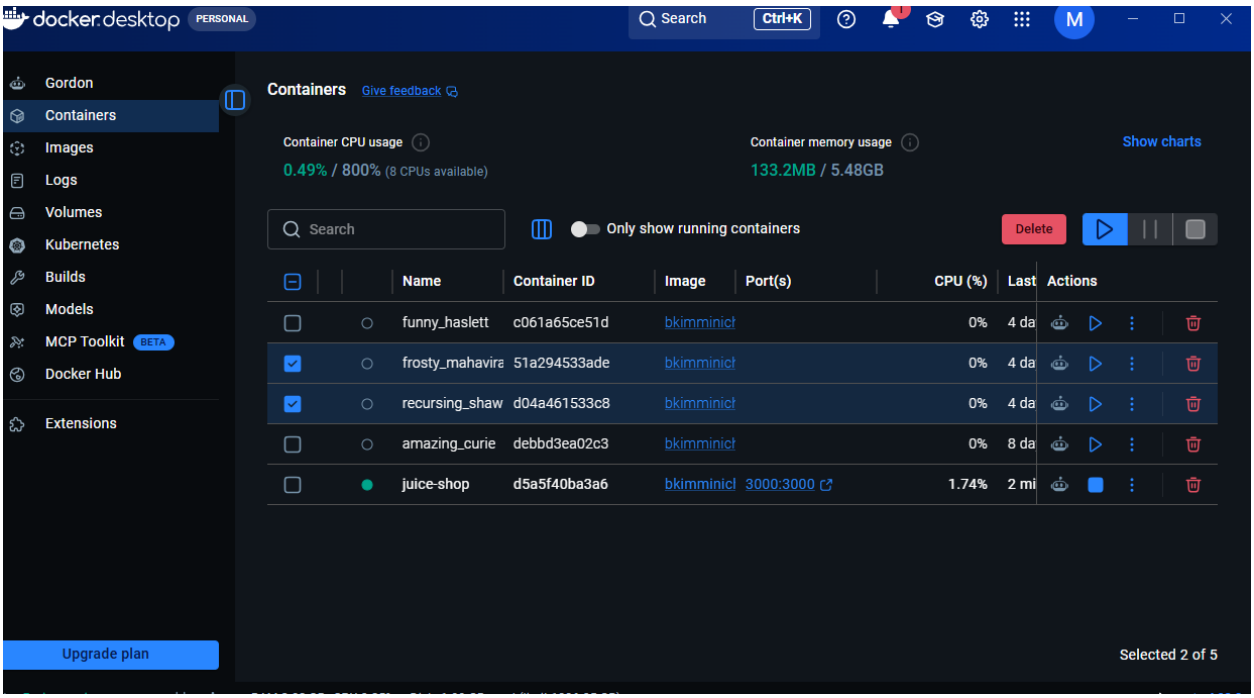



*Figure 2: Docker Container Status for the Local Testing Environment.*

The Docker output provides evidence of the running container and its port configuration. The actual container name, image and port mapping should be recorded from the command output.

## 4. Identification of Application Entry Points

Application entry points are pages, forms, fields and features through which a user interacts with an application. Identifying these entry points helps establish the application's attack surface for later security assessment.

### 4.1 Home Page

The home page was inspected to identify the application's main features and navigation options. Features available from the interface provide starting points for examining how the application handles user interactions.

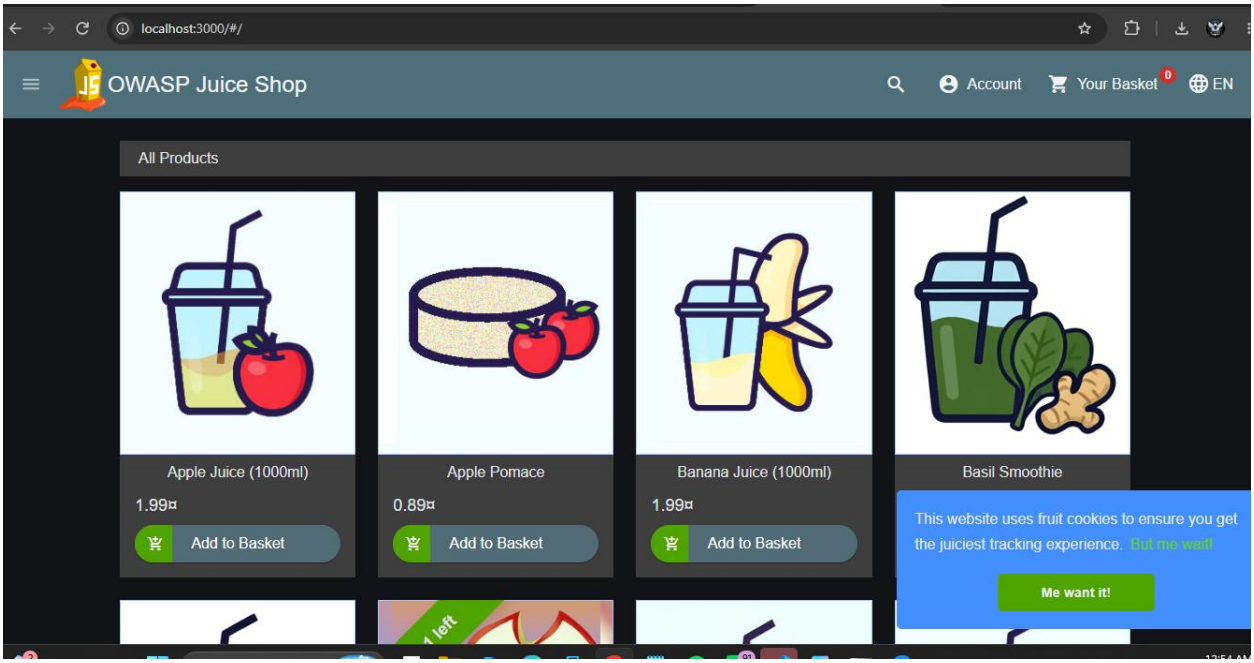



*Figure 3: Initial Identification of OWASP Juice Shop Entry Points.*

The home page was used to identify features for further investigation, including product search, login, registration and the shopping basket.

### 4.2 Product Search

The product search feature was tested by entering a search term and monitoring the resulting request in Chrome Developer Tools.

A GET request was observed during the search operation. The corresponding request URL and response details should be captured to establish the exact endpoint and the behavior of the search feature.

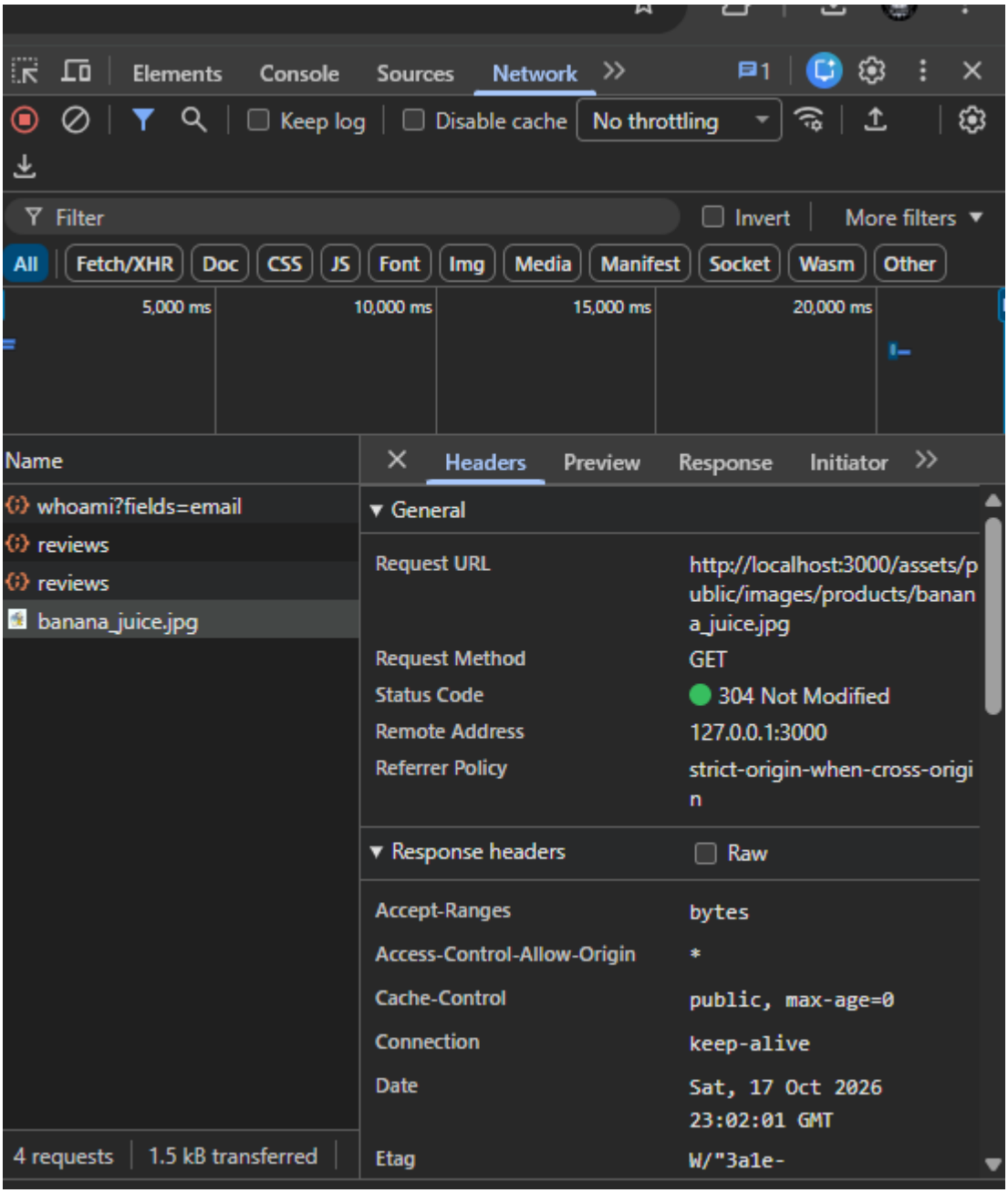



*Figure 4: HTTP GET Request Generated by the Product Search Feature.*

**Observation:** The search feature generated an HTTP GET request. This provided an initial indication of how the frontend communicates with the backend to retrieve search-related information.

### 4.3 Login Form

The login form was examined by submitting a test login attempt in the local lab environment. A POST request was observed, and the application displayed an error message.

The exact request URL, response status and response behaviour should be documented using the captured network evidence.

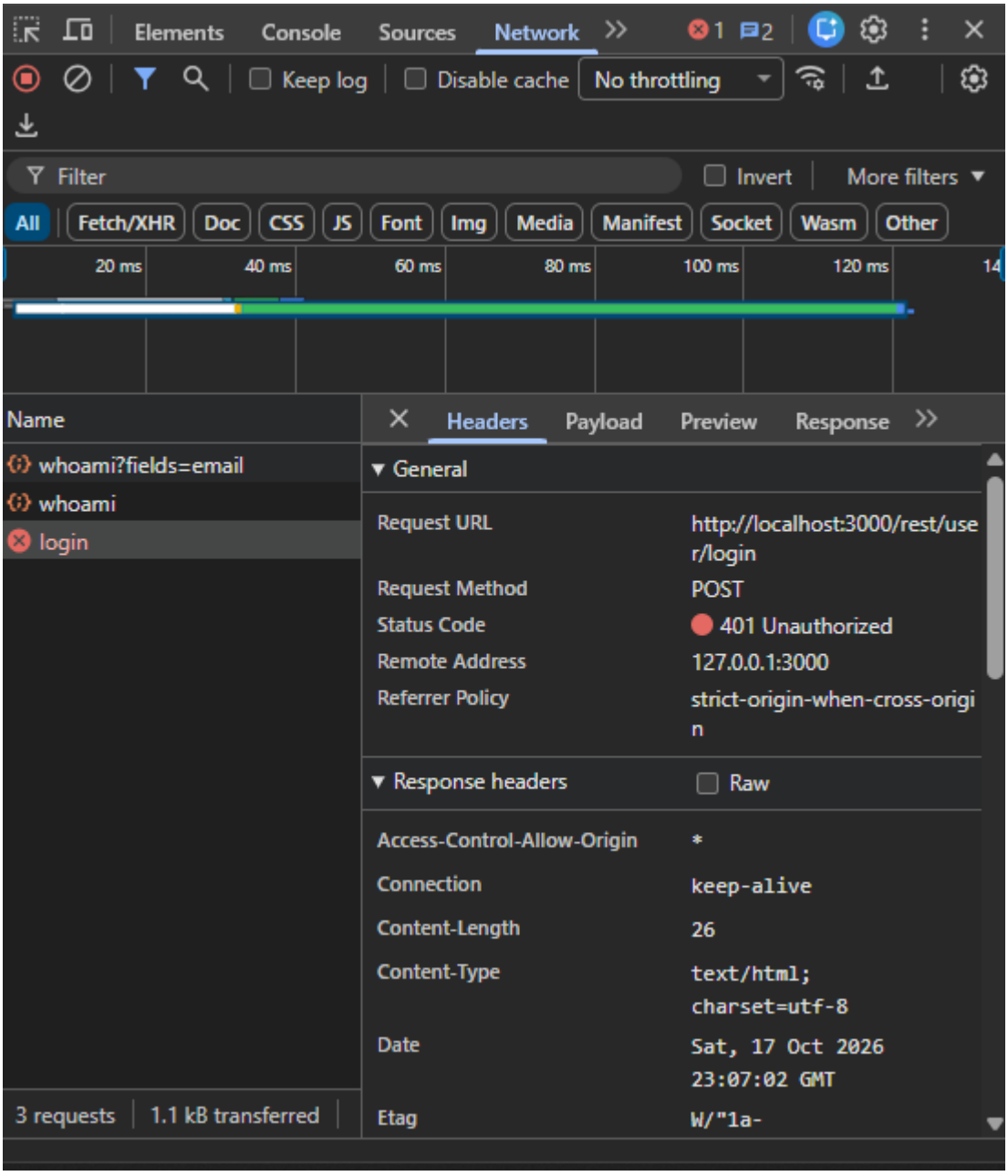



*Figure 5: HTTP POST Request Generated by the Login Form.*

**Observation:** The login operation generated a POST request. This identified an authentication-related entry point for further assessment. The error message alone does not demonstrate that the authentication mechanism is vulnerable.

### 4.4 Registration Form

The registration page will be inspected to identify the fields that accept user information and the request used to submit registration details.

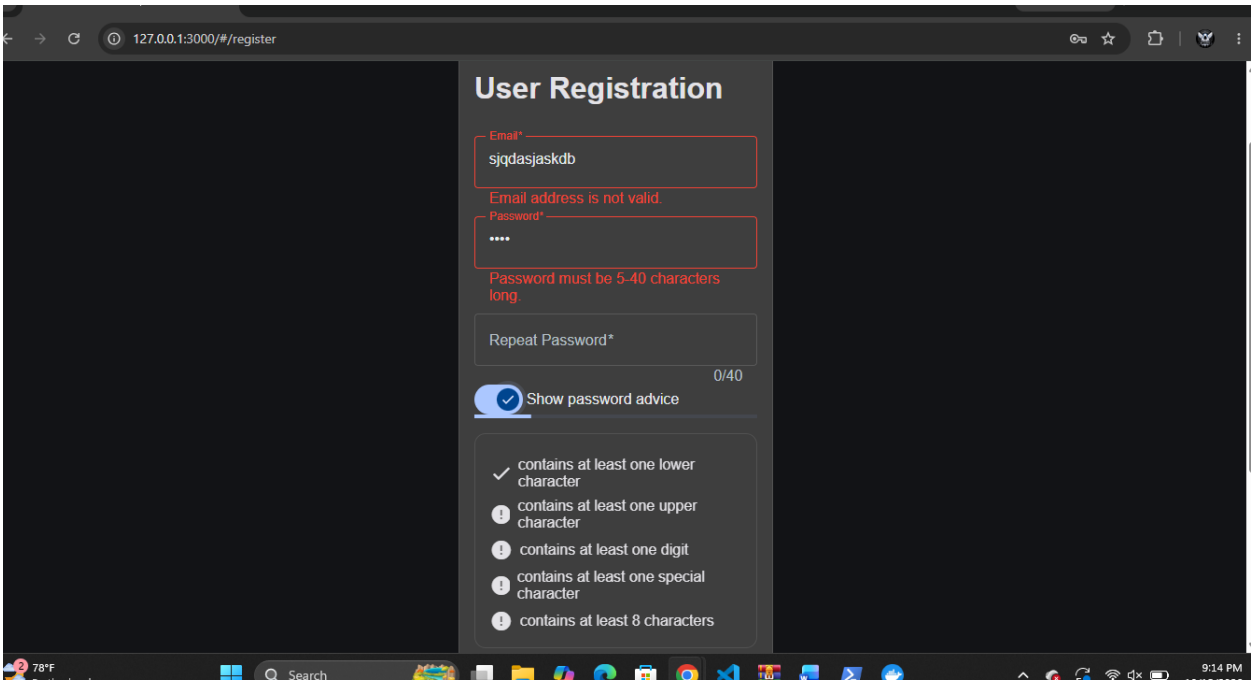



*Figure 6: Registration Form and Identified User Input Fields.*

The final observation should list the fields actually displayed in the running application and describe the corresponding request if it has been captured.

### 4.5 Shopping Basket

The basket feature will be inspected to identify requests associated with retrieving or modifying basket information. This feature is relevant to understanding how the application manages user-specific data and shopping operations.

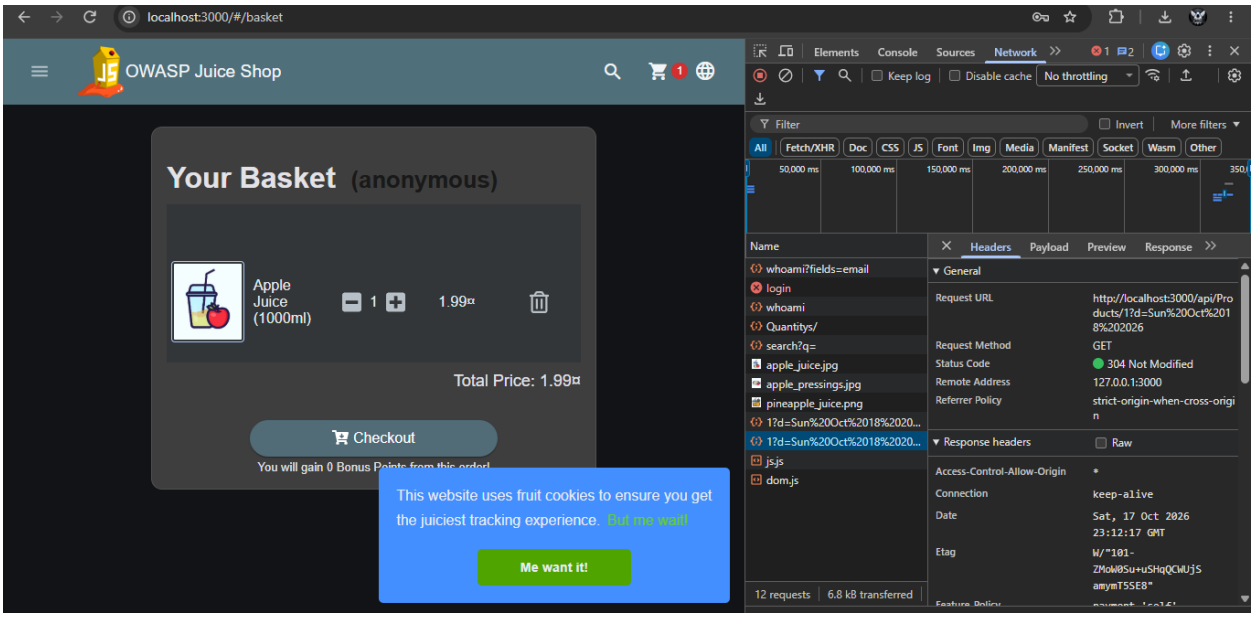



*Figure 7: Shopping Basket Functionality and Related API Requests.*

The final findings displayed observed HTTP method, exact endpoint and response status. Any access-control conclusions would require additional evidence and testing.

### 5. Technology Stack Identification

The technology stack describes the technologies used to develop and run the application, including the frontend, backend, server environment and supporting components.

The initial inspection confirmed that the application was accessed through a browser and run using Docker. Additional details should be verified through container information, response headers, application assets and reliable project documentation.

The following table separates the current observations from details that still require confirmation.

| Component | Current finding | Verification method |
|---|---|---|
| Operating system | Windows | System environment |
| Container platform | Docker | Docker commands |
| Frontend framework | To verify for the running version | Application assets and project documentation |

| | | |
|---|---|---|
| Backend runtime | To verify for the running version | Container image and project documentation |
| HTTP communication | GET and POST observed | Chrome Developer Tools |
| API interface | HTTP endpoints observed during application interactions | Network request inspection |
| Web server details | Not yet confirmed | Response headers and container configuration |

OWASP Juice Shop is commonly associated with an Angular frontend and a Node.js backend. These technologies should only be recorded as confirmed findings after checking documentation or evidence applicable to the version being tested.

**6. Application Structure**

The application was examined from the perspective of its user interface, HTTP communication and backend functionality.

At a high level, the application can be represented as follows:

1. **Frontend:** Provides the user interface for browsing products, searching, authentication, registration and basket operations.
2. **HTTP communication layer:** Transfers requests from the browser to application endpoints and returns responses.
3. **Backend:** Processes incoming requests and implements application functionality.
4. **Data layer:** Supports the storage and retrieval of application information.

This is a conceptual representation of a typical web application architecture. The internal implementation of each component should be confirmed through appropriate evidence before making more detailed architectural claims.

**7. API Endpoint Inventory**

An API endpoint inventory records the requests observed during reconnaissance. It helps establish which parts of the application communicate with the backend and provides a reference for subsequent assessment.

The following table is an initial inventory.

| **Feature** | **HTTP method** | **Exact endpoint** | **Response status** |
|---|---|---|---|
| Product search | GET | http://localhost:3000/assets/public/images/products/apple_juice.jpg | 304 |
| Login | POST | http://localhost:3000/rest/user/login | 401 |
| Registration | POST | http://localhost:3000/api/SecurityQuestions/ | 304 |
| Basket | POST | http://localhost:3000/api/Quantitys/ | 304 |

The HTTP method and endpoint should be copied from the selected request's Headers tab. The response status should be taken from the actual response rather than inferred from the message displayed on the page.

Query parameters and request bodies should be documented without including passwords, session cookies, authentication tokens or other sensitive values.

**8. Custom Python Scanner Findings**

A custom Python scanner was developed to support the reconnaissance process. The scanner generated a JSON report containing information about pages, forms, input points and potential API endpoints.

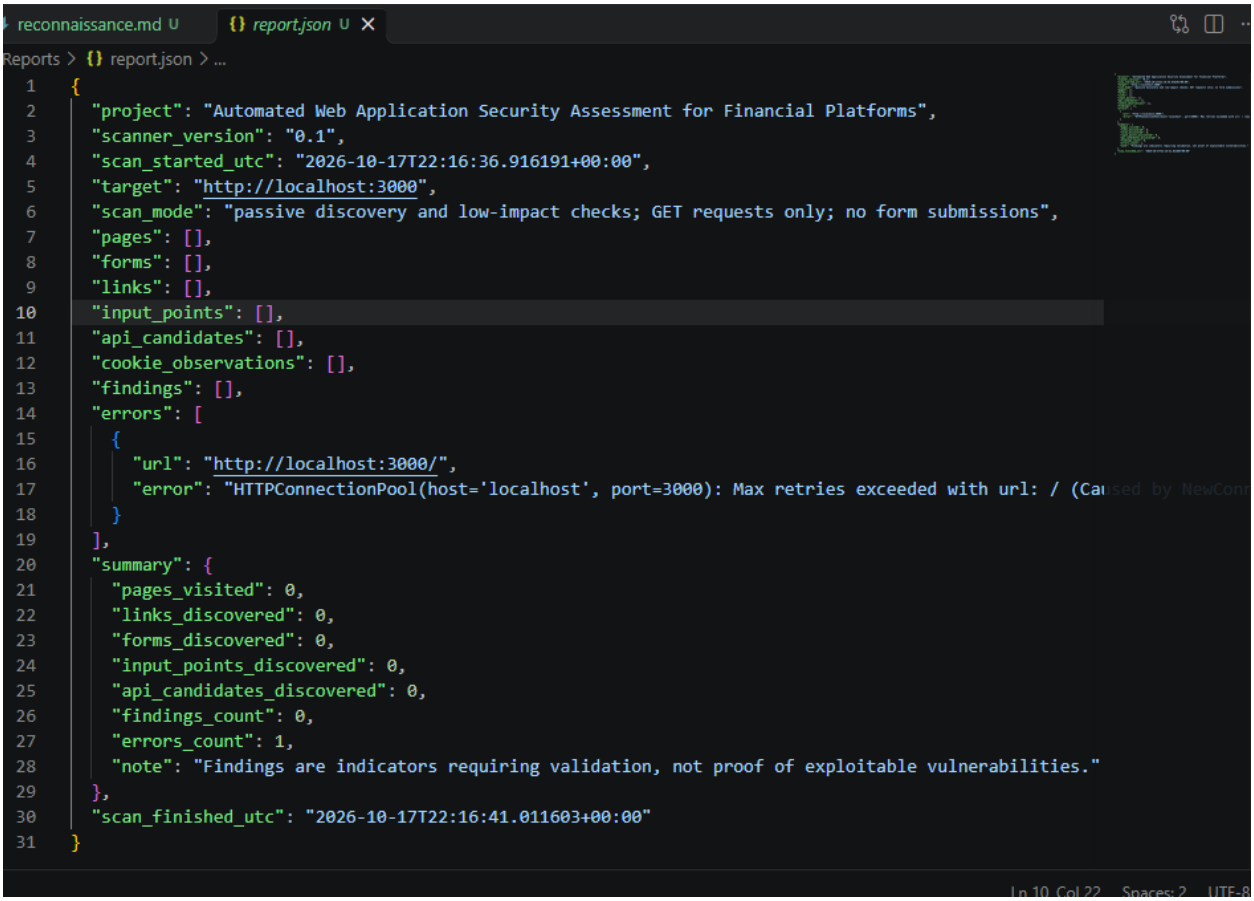



*Figure 8: JSON Output Generated by the Python Reconnaissance Scanner.*

The report provides a structured record of the information collected by the scanner. Its endpoint candidates can be compared with the requests observed manually in the browser.

A scanner-generated candidate should not automatically be treated as a confirmed API endpoint or a vulnerability. Each relevant finding should be checked against the actual application and its observed behavior.

**9. Relevance to Financial Platforms**

The reconnaissance findings are relevant to the planned assessment of financial web applications because banking and financial platforms commonly provide user-facing features such as authentication, registration, information retrieval and account-related operations.

The features examined in Juice Shop provide a starting point for considering similar application entry points in financial systems.

For example:

- **Authentication:** Financial applications must protect customer accounts against unauthorised access.
- **Input handling:** Search fields and forms should validate and process user input appropriately.
- **API security:** Backend endpoints should enforce appropriate authentication, authorisation and input validation.
- **Session management:** Sensitive account operations should be protected by appropriate session controls.
- **Data protection:** Requests and responses should not expose sensitive customer information unnecessarily.

These are areas for future assessment, not claims that the corresponding vulnerabilities have been found in Juice Shop or in a real financial platform.

**10. Limitations**

The reconnaissance phase focused on identifying application entry points and observing normal application behaviour.

The work does not constitute a complete security audit, penetration test or vulnerability verification exercise. Some technology details and API endpoints may remain unverified until further inspection.

The scanner's output is intended to support reconnaissance and should not be interpreted as proof of exploitable vulnerabilities.

Testing was limited to the authorised local lab environment. No exploit payloads or unauthorised testing of real financial platforms were required for this phase.

**11. Conclusion**

The initial reconnaissance of OWASP Juice Shop established a foundation for the Automated Web Application Security Assessment for Financial Platforms project. The product search feature

generated a GET request, while the login operation generated a POST request. These observations provided an initial understanding of how the application's frontend communicates with its backend.

The custom Python scanner also generated a JSON report containing pages, forms, input points and API candidates.

The remaining work is to complete the inspection of registration and basket functionality, verify the relevant technology stack, document exact API endpoints and compare manual observations with the scanner's output. The completed findings will guide the next development stage of the security assessment tool.